Árvores, Random Forest e Boosting

ÁRVORE / FLORESTA

Árvore

```
from sklearn.tree import DecisionTreeClassifier
arv=DecisionTreeClassifier(
 max_depth=4,random_state=42)
arv.fit(Xtr,ytr)
```

Árvores não exigem scaler.

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
rf=RandomForestClassifier(
 n_estimators=200,min_samples_leaf=3,
 random_state=42,n_jobs=-1)
rf.fit(Xtr,ytr)
```

Bagging reduz variância.

BOOSTING

HistGradientBoosting

```
from sklearn.ensemble import (
 HistGradientBoostingClassifier)
hgb=HistGradientBoostingClassifier(
 max_iter=150,learning_rate=.05,
 random_state=42)
hgb.fit(Xtr,ytr)
```

Boosting é sequencial.

Comparar CV

```
from sklearn.model_selection import cross_val_score
for m in (arv,rf,hgb):
 print(cross_val_score(
 m,Xtr,ytr,cv=5).mean())
```

Mesma métrica/folds para todos.

INTERPRETAR

Importâncias árvore

```
print(rf.feature_importances_)
```

Impureza pode enviesar rankings.

Permutação no teste

```
from sklearn.inspection import (
 permutation_importance)
r=permutation_importance(rf,Xte,yte,
 n_repeats=5,random_state=42)
```

Atributos correlatos complicam leitura.

Ajustar complexidade

```
print(arv.get_depth())
```

Controle profundidade/folhas.

Receitas de consulta: adapte dados, colunas, credenciais e versões. Execute somente em ambientes autorizados.